

INF01151 – Sistemas Operacionais II

Aula 08 - Da teoria à prática... Parte II
(Processos, threads, mutex, semáforos,
variáveis de condição e monitores)

Prof. Alberto Egon Schaeffer Filho

Mecanismos de sincronização

	UNIX	POSIX Threads	Java
Semáforos	Sim	Não ¹	Sim ²
Mutex (lock/unlock)	Não	Sim	Sim ²
Variáveis de condição	Não	Sim	Sim ^{2,3}
Monitores	Não	Não	Sim

¹ Não fazem parte da especificação *POSIX Threads* (norma POSIX 1003.1c), mas é possível empregar os semáforos Unix (norma POSIX 1003.1b)

² API específica disponível desde a versão 1.5 através do pacote *java.util.concurrent*

³ Possui uma variável de condição implícita por objeto sincronizado

Semáforos Unix

Existem dois tipos: *unnamed* e *named*

- **Unnamed**: existe no espaço de endereçamento de processos ou threads (*process-wide*)
 - Usados para sincronizar threads e processos relacionados
 - Declarado como variável global: *pshared* = 0 para threads, *pshared* ≠ 0 para processos
 - Não persistentes, existem apenas enquanto os processos executam
 - Se programador esquecer de fechar semáforo, ele é zerado quando processo termina
- **Named**: existe no sistema de arquivos, i.e., é um descritor de arquivo (*system-wide*)
 - Uso similar ao de um arquivo
 - Permite sincronização entre processos, relacionados ou não, e threads
 - Threads e processos usam da mesma forma
 - Persistentes (existem até o *shutdown* do sistema)
 - Limitado em número pelo SO

API para semáforos *named*

(sistema de arquivos)

```
#include <sys/semaphore.h>
```

```
sem_t *sem_open (const char *name, int oflag, mode_t mode, int value);
```

- *name*: caminho de um arquivo (semáforo)

- *oflag*: modo de abertura do semáforo

- O_CREAT para criar um semáforo se ele não existe, O_EXCL se a chamada deve falhar se já existente

- *mode*: permissões do arquivo semáforo

- *value*: valor inicial do semáforo

Chamada que cria o semáforo,
inicializa e configura as permissões



```
int sem_wait (sem_t *sem); /* equivalente a primitiva P */
```

```
int sem_trywait (sem_t *sem); /* adquire semáforo se livre, caso  
contrário retorna sem adquirir */
```

```
int sem_post(sem_t *sem); /* equivalente a primitiva V */
```

```
int sem_close(sem_t *sem); /* encerra o uso do semáforo */
```

```
int sem_unlink(const char *name); /* remove do sistema */
```

API para semáforos *unnamed*

```
#include <semaphore.h>
```

```
int sem_init (sem_t * sem, int pshared, unsigned int value);
```

- *sem*: variável do tipo semáforo
- *pshared*: visibilidade intra processo (= 0) ou inter processos (≠ 0)
- *value*: valor inicial do semáforo

```
int sem_wait (sem_t *sem);          /* equivalente a primitiva P */
```

```
int sem_trywait (sem_t *sem);       /* adquire semáforo se livre, caso  
                                     contrário retorna sem adquirir */
```

```
int sem_post(sem_t *sem);           /* equivalente a primitiva V */
```

```
int sem_destroy(sem_t *sem);        /* destruição do semáforo */
```

Exemplo de uso semáforos *unnamed*

```
#include <pthread.h>
#include <semaphore.h>
#include <stdlib.h>
#include <stdio.h>

#define N      2
#define TRUE  1

int      buffer[N], in=0, out=0;
sem_t    vazio, cheio, mutexP, mutexC;

void *produtor(void *arg) {

    while(TRUE) {
        sem_wait(&vazio);
        sem_wait(&mutexP);
        buffer[in] = rand() % 100;
        printf("P %d\n", buffer[in]);
        in= (in+1) % N;
        sem_post(&mutexP);
        sem_post(&cheio);
    }
}
```

```
void *consumidor(void *arg) {
    int item;
    while(TRUE) {
        sem_wait(&cheio);
        sem_wait(&mutexC);
        item = buffer[out];
        printf("Consumindo %d\n", item);
        out = (out+1) % N;
        sem_post(&mutexC);
        sem_post(&vazio);
    }
}
```

```
int main(int argc, char *argv[ ]) {
    pthread_t cons, prod;
    sem_init(&mutexP, 0, 1);
    sem_init(&mutexC, 0, 1);
    sem_init(&vazio, 0, N);
    sem_init(&cheio, 0, 0);
    pthread_create(&prod, NULL, produtor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
}
```

Sincronização com POSIX threads

- Podem usar semáforos Unix
- Variáveis do tipo mutex (sincronização de acesso)
- Variáveis de condição (sincronização de controle)

POSIX threads: *mutex locks*

- É um tipo de dados abstrato (**pthread_mutex_t**)
 - Necessita ser declarado e inicializado
- Duas formas de inicializar
 - Na declaração: **pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;**
 - Dinamicamente, com **pthread_mutex_init(&mutex, &attr);**
- Atributos do tipo **pthread_mutexattr_t**, ou **NULL** para *defaults*:
 - **Protocol**: especifica protocolo usado para evitar inversão de prioridades
 - **Prioceiling**: especifica o teto de prioridade do mutex
 - **Process-shared**: especifica se o mutex é compartilhado em processos

- Semântica mais simples que semáforos
- Somente a thread que fez **lock** pode fazer **unlock**

```
pthread_mutex_t mutex;  
pthread_mutex_init(&mutex, NULL);  
...  
pthread_mutex_lock(&mutex);  
{seção crítica}  
pthread_mutex_unlock(&mutex);
```

Inicialização simplificada, utilizando mutex com os atributos default

API para mutex locks (POSIX)

```
#include <pthread.h>
```

```
int pthread_mutex_init (pthread_mutex_t *mutex,  
                        const pthread_mutexattr_t* attr );
```

```
int pthread_mutex_lock ( pthread_mutex_t *mutex );  
int pthread_mutex_trylock ( pthread_mutex_t *mutex );  
int pthread_mutex_unlock ( pthread_mutex_t *mutex );
```

```
int pthread_mutex_destroy (pthread_mutexattr_t *mutex );
```

Obtém o lock se estiver livre, senão bloqueia

Tenta obter o lock. Se não conseguir, retorna com código de erro “busy”

Libera o lock. Gera código de erro se lock já estava livre ou se lock pertence a outra thread

Exemplo de uso mutex locks (POSIX)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define TRUE 1

pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void *minha_funcao (void *arg) {
    int n = *(int *) arg;
    while(TRUE) {
        pthread_mutex_lock(&m);
        printf("Thread %d iniciando a SC\n ", n);
        printf("Thread %d ainda na SC\n ", n);
        printf("Thread %d saindo da SC\n ", n);
        pthread_mutex_unlock(&m);
    }
}

int main (int argc, char *argv[]) {
    pthread_t  thr1, thr2;
    int n1 = 1; int n2 = 2;

    pthread_create(&thr1, NULL, minha_funcao, (void *) &n1);
    pthread_create(&thr2, NULL, minha_funcao, (void *) &n2);

    pthread_exit(0);
}
```

Variáveis de condição: POSIX threads

- Poderoso mecanismo de sincronização condicional
 - *Mutex*/semáforo binário apenas permitem que *threads* ou processos serializem seus acessos a uma seção crítica
 - Permite que thread bloqueie se condição não for verdadeira
 - E que outra thread sinalize quando condição ocorrer
 - Uso típico é informar o estado de dados compartilhados
 - ex.: buffer está vazio, buffer está cheio, que evento desejado ocorreu etc.
 - Deve ser sempre **usada com um *mutex*!**
 - Necessário garantir acesso exclusivo a dado p/ consultar seu valor
 - Thread que deseja fazer wait deve primeiro obter o lock
- ```
pthread_mutex_lock(&mutex);
```
- E depois executar
- ```
pthread_cond_wait(&cond, &mutex);
```
- Faz com que a thread libere o lock e espere na variável de condição

Variáveis de condição: POSIX threads

- Tipo de dados abstrato (**pthread_cond_t**)
 - Necessita ser declarado e inicializado
- Duas formas de inicializar
 - Na declaração: **pthread_cond_t mycond = PTHREAD_COND_INITIALIZER;**
 - Dinamicamente: **pthread_cond_init (&mycond, &attr);**
- Mais fácil compreensão e uso que semáforos
 - Espera por um evento: **cond_wait**
 - Sinalizando a ocorrência de um evento: **cond_signal** e **cond_broadcast**
 - “Acorda” UMA (*cond_signal*) ou TODAS (*cond_broadcast*) *threads*
 - Importante: as *threads* não memorizam *signal* (diferente de semáforos)
- Sempre devem ser usadas em conjunto com um **mutex**!
 - Liberado automaticamente e atomicamente em **cond_wait**
 - Obtido automaticamente e atomicamente em **cond_signal**

Um único atributo:
process-shared

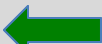
Variáveis de condição: POSIX threads

- Sequência típica (receita) de uso:

Thread Main

Declarar e inicializar **variáveis globais** que necessitam sincronização (p. ex. count, buffer, etc)
Declarar e inicializar **váriavel de condição**
Declarar e inicializar **mutex lock**
Criar threads **A** e **B**

Thread A

Executar até ponto em que condição deve ocorrer
Obter o **mutex**
Verificar valor da variável global (p.ex. count == n)
Se necessário, **bloquear** pthread_cond_wait() 
→ mutex automaticamente liberado
Acordar quando for sinalizado 
→ mutex automaticamente obtido
Liberar o **mutex**

Thread B

Executar
Obter o **mutex**
Modificar valor da variável global que Thread A
está esperando
Se valor satisfazer condição da Thread A, **sinalizar**
→ semântica *signal and continue*
Liberar o **mutex** 

Thread Main

Esperar até que threads **A** e **B** terminem (*join*)

API POSIX para variáveis de condição

```
#include <pthread.h>
```

```
int pthread_cond_init (pthread_cond_t *cond, const pthread_condattr_t *attr);  
int pthread_condattr_init(pthread_condattr_t *cond);
```

```
int pthread_cond_wait (pthread_cond_t *cond, pthread_mutex_t *mutex);  
int pthread_cond_signal (pthread_cond_t *cond);  
int pthread_cond_broadcast (pthread_cond_t *cond);
```

```
int pthread_cond_destroy (pthread_cond_t *cond);
```

Exemplo de uso variáveis de condição

```
#include <pthread.h>
#include <stdio.h>
#define TRUE 1
#define MAX_ITEMS 10

pthread_cond_t not_full, not_empty;
pthread_mutex_t mutex;

int count = 0, in = 0, out = 0;
int buffer[MAX_ITEMS];

void *produtor(void *arg) {

    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == MAX_ITEMS)
            pthread_cond_wait(&not_full, &mutex);
        buffer[in] = rand() % 5;
        count++;
        printf("P buffer[%d]=%d\n", in,
            buffer[in]);
        in = (in + 1) % MAX_ITEMS;
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
    }
}
```

```
void *consumidor(void *arg) {
    int task;
    while (TRUE) {
        pthread_mutex_lock(&mutex);
        while (count == 0)
            pthread_cond_wait(&not_empty, &mutex);
        task = buffer[out];
        count--;
        printf("C buffer[%d] = %d\n", out, task);
        out = (out + 1) % MAX_ITEMS;
        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
    }
}

int main(int argc, char *argv[]) {
    pthread_t prod, cons;
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);
    pthread_mutex_init(&mutex, NULL);
    pthread_create(&prod, NULL, produtor, NULL);
    pthread_create(&cons, NULL, consumidor, NULL);

    pthread_exit(0);
}
```

Monitores em Java

- Fornece exclusão mútua em um objeto
 - Emprego da palavra-chave *synchronized*
 - Utilizada com um método inteiro
 - Utilizada com uma sequência de comandos

```
class Interfere {  
    private int data = 0;  
    public void update( ) {  
        data++;  
    }  
}
```



Se método **update** for chamado simultaneamente por duas threads, elas vão causar interferência nas suas execuções (seção crítica)

```
class Interfere {  
    private int data = 0;  
    public synchronized void update() {  
        data++;  
    }  
} // método sincronizado
```

```
class Interfere {  
    private int data = 0;  
    public void update() {  
        synchronized(this) {  
            data++;  
        }  
    }  
} // bloco sincronizado
```


Monitores em Java

- Como monitores são programados em Java
 - Variáveis permanentes do monitor são os dados privados da classe
 - Procedimentos são implementados como métodos **synchronized**
 - Exclusão mútua “implícita”, um lock por objeto em Java
 - Quando método sincronizado é invocado, ele espera pelo lock, executa, e libera o lock
- Primitivas para sincronização de condição
 - Precisam ser executados em porções de código **synchronized**
 - **wait**: thread que executa fica bloqueada esperando por um notify (signal)
 - Libera o monitor
 - **notify**: acorda uma única thread que está bloqueado na fila de espera
 - **notifyAll**: acorda todas as threads da fila de espera
 - Não suporta diretamente variáveis de condição, mas pode-se pensar que existe uma variável de condição (implícita) por objeto sincronizado

Monitores em Java: comentários

- Existe uma única fila por objeto sincronizado
 - A política da fila não é padronizada, mas geralmente é FIFO
- Java implementa a semântica ***signal-and-continue (SC)***
 - Como a *thread* que sinaliza continua a executar, pode acontecer que a condição sinalizada não seja mais verdadeira
 - *Thread* que foi “acordada” para entrar no monitor pode ter que voltar a esperar a condição
- Permite chamadas aninhadas, na forma ***closed call***
 - Necessário cuidar com deadlocks
- Códigos-exemplos:
 - http://wps.prenhall.com/br_deitel_sistemas_3

Leituras adicionais

- Andrews, G. *“Foundations of Multithreaded, Parallel and Distributed Programming”*, Addison-Wesley, 2000.
 - Capítulo 4 (seção 4.6) e capítulo 5 (seção 5.4 e 5.5)
- Silberschatz, A. and Galvin, P. – *“Operating Systems Concepts”*. Wiley, 8th edition, 2009.
 - Capítulo 4 (seção 4.3)
- Butenhof, D. – *“Programming with POSIX Threads”*. Addison-Wesley. 1997.